PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Циклы: повторение действий в алгоритмах

Содержание

- Зачем нужен цикл
- Цикл for и функция range()
- Цикл while
- Счётчик, накопитель, максимум
- Управление циклом: break и continue
- Когда использовать for, а когда while
- Типичные затруднения
- Итоги урока
- Материалы к занятию

Зачем нужен цикл

Представьте, что нужно 100 раз подряд вывести одно и то же сообщение или сложить 1000 чисел. Писать 100 строк `print()` — не выход: программа станет длинной, а ошибку придётся исправлять в каждой копии.

Цикл — это конструкция, которая повторяет действие нужное число раз или пока выполняется условие. Повторяемое действие записывается один раз, а цикл управляет количеством повторений.

На этом занятии программы используют только переменные, `if/elif/else`, `for`, `while`, `input()` и `print()` — без списков и функций. Значения обрабатываются по одному: программа хранит не весь набор, а только текущее состояние — сколько значений подошло, их сумму, наибольшее найденное.

Цикл for и функция range()

Цикл `for` используют, когда заранее известно, сколько раз нужно повторить действие. Функция `range()` задаёт последовательность чисел, которые по очереди принимает переменная цикла.

Запись	Что означает
`range(5)`	0, 1, 2, 3, 4 — пять значений, начиная с нуля
`range(1, 6)`	1, 2, 3, 4, 5 — от 1 до 5 включительно
`range(1, 10, 2)`	1, 3, 5, 7, 9 — от 1 до 9 с шагом 2
`range(10, 0, -1)`	10, 9, 8 … 1 — обратный отсчёт

Правая граница в `range()` не входит в последовательность: `range(1, 6)` заканчивается на 5. Функция `range()` создаёт последовательность **целых** чисел; аргументы тоже должны быть целыми: `range(2.5)` вызовет ошибку `TypeError`. Если начало не меньше конца (при положительном шаге), последовательность получится пустой и цикл не выполнится ни разу: `range(5, 1)` и `range(0)` не дают ни одного значения.

```
# Вывести числа от 1 до 5
for i in range(1, 6):
    print(i)

# Сложить числа от 1 до 10
total = 0
for i in range(1, 11):
    total = total + i
print('Сумма:', total)
```

Цикл while

Цикл `while` повторяет действие, **пока условие истинно**. Чаще всего его выбирают, когда число повторений заранее неизвестно: оно зависит от ввода или от того, как меняются данные. Это удобное правило, но не строгий запрет — любой цикл `for` можно записать через `while`, и наоборот. Выбирайте тот цикл, который проще и понятнее описывает задачу.

```
# Считать числа, пока не введут 0
number = int(input('Введите число (0 – стоп): '))
while number != 0:
    print('Вы ввели:', number)
    number = int(input('Введите число (0 – стоп): '))
print('Конец ввода')
```

Запись `int(input())` ожидает, что пользователь введёт целое число. Если ввести текст или дробное число (например, «пять» или 2.5), программа остановится с ошибкой `ValueError`. Поэтому в задачах вводите данные в том формате, который указан в условии.

> **Осторожно: бесконечный цикл**
>
> Если условие `while` никогда не станет ложным, программа зависнет. Проверьте, что внутри цикла меняется переменная, от которой зависит условие. Остановить зависшую программу можно кнопкой остановки в среде разработки или блокноте, в терминале — сочетанием клавиш Ctrl+C. Надёжнее не допускать зависания: перед запуском убедитесь, что значение в условии может измениться.

Счётчик, накопитель, максимум

Большинство задач с циклами решаются тремя приёмами. Для каждого важно задать начальное значение **до** цикла.

Приём	Отвечает на вопрос	Начальное значение	Обновление в цикле
Счётчик	Сколько значений подошло?	`count = 0`	`count = count + 1`
Накопитель	Какова сумма значений?	`total = 0`	`total = total + x`
Максимум	Какое значение наибольшее?	первое обработанное значение	`if x > max_val:` `max_val = x`
Минимум	Какое значение наименьшее?	первое обработанное значение	`if x < min_val:` `min_val = x`

Начинать максимум с нуля нельзя: если все числа отрицательные, ни одно не окажется больше нуля, и в ответе останется 0, которого в наборе не было.

Если значений нет совсем. Максимум и минимум начинают с первого обработанного значения. Но если пользователь сразу ввёл стоп-значение, обрабатывать нечего и максимум не определён. Поэтому перед выводом проверяют, было ли хотя бы одно значение:

```
count = 0
x = int(input('Введите число (0 – стоп): '))
while x != 0:
    count = count + 1
    if count == 1 or x > max_val: # первое значение или новое наибольшее
        max_val = x
    x = int(input('Введите число (0 – стоп): '))

if count > 0:
    print('Максимум:', max_val)
else:
    print('Числа не были введены')
```

Управление циклом: break и continue

Оператор	Что делает	Когда нужен
`break`	Сразу завершает цикл	Нужное значение найдено, дальше проверять не нужно
`continue`	Пропускает остаток тела цикла и переходит к следующему шагу	Текущее значение не подходит, например нечётное

```
# break: выйти из цикла при вводе 0
while True:
    x = int(input('Введите число: '))
    if x == 0:
        break
    print('Число:', x)
print('Цикл завершён')
```

После `break` программа продолжает работу с первой строки **после** цикла — в примере выше это `print` с сообщением о завершении. Если один цикл находится внутри другого, `break` завершает только тот цикл, в котором он записан; внешний цикл продолжит работу.

Пример с `continue`: вывести только чётные числа от 1 до 10. Для нечётного числа `continue` пропускает `print` и сразу переходит к следующему числу.

```
for i in range(1, 11):
    if i % 2 == 1: # остаток от деления на 2 равен 1 — число нечётное
        continue
    print(i)
# Выведет: 2 4 6 8 10 (каждое число на новой строке)
```

Когда использовать for, а когда while

Ситуация	Цикл
Известно количество повторений (5 раз, 10 раз)	`for`
Нужно перебрать диапазон чисел	`for` + `range()`
Повторять, пока выполняется условие	`while`
Повторять до ввода определённого значения	`while`
Неизвестно, сколько значений введёт пользователь	`while`

Типичные затруднения

> **Если что-то не получается**
>
> **«Не хватает последнего числа»** — правая граница `range()` не включается: для чисел от 1 до N пишите `range(1, N + 1)`.
> **NameError: name 'total' is not defined** — накопитель не задан до цикла. Добавьте `total = 0` перед циклом.
> **Счётчик на единицу больше или меньше** — проверьте начальное значение и место, где счётчик увеличивается. Обычно счётчик начинают с 0 и увеличивают только тогда, когда значение подошло под условие.
> **Максимум равен 0, хотя все числа отрицательные** — начните максимум с первого введённого числа.
> **Программа зависла** — в `while` не меняется переменная из условия или недостижим `break`.

> **Вывод «съехал»** — проверьте отступы: команда, записанная с отступом тела цикла, повторяется на каждом шаге.

Итоги урока

- Цикл `for` с `range()` перебирает известное количество значений; правая граница `range()` не включается.
- Цикл `while` повторяет действие, пока условие истинно; внутри цикла условие должно когда-то стать ложным.
- Счётчик и накопитель начинаются с нуля, максимум и минимум — с первого значения; если значений может не быть, это нужно проверить.
- `break` завершает цикл, `continue` переходит к следующему шагу.

Материалы к занятию

- 17.pptx (14.7 МБ)